

GitHub and Postman API Assignment Report

1. Introduction

In this assignment, I used GitHub's REST API through Postman to perform different operations on a GitHub repository. The purpose of the assignment was to understand how APIs work and how different HTTP methods can be used to interact with GitHub resources.

I used Postman to send requests to the GitHub API and verify the responses. I worked with authentication, repositories, files, and issues. I also used environment variables and the Collection Runner to make the requests easier to manage and test.

The main HTTP methods I used were GET, POST, PUT, PATCH, and DELETE.

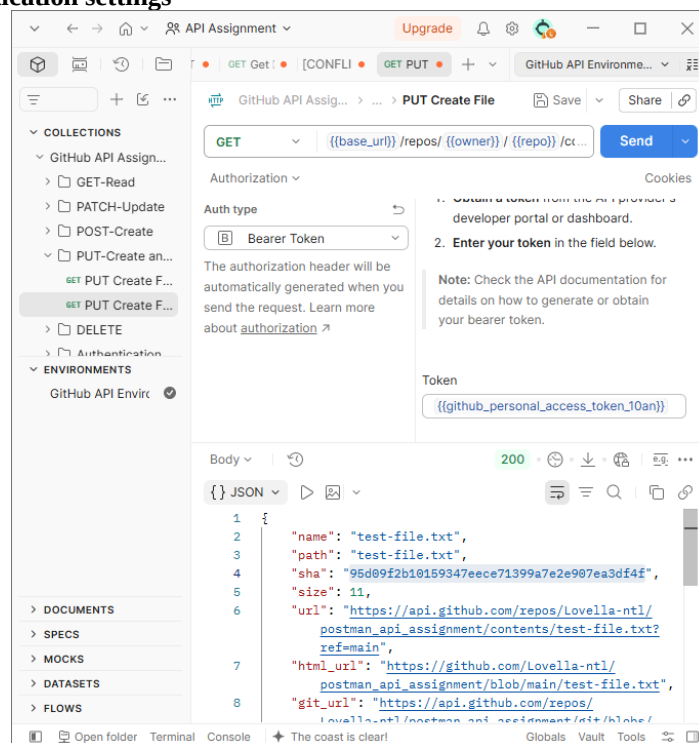
2. Authentication

The first step was to authenticate my requests so that GitHub could identify and authorize me. I used a GitHub Personal Access Token for authentication in Postman.

I configured the authentication settings in my Postman request and selected the appropriate authentication method. The token was added securely rather than repeatedly typing it into every request.

Authentication was important because some GitHub API operations require permission to access or modify repository information.

Screenshot 1 – Authentication settings



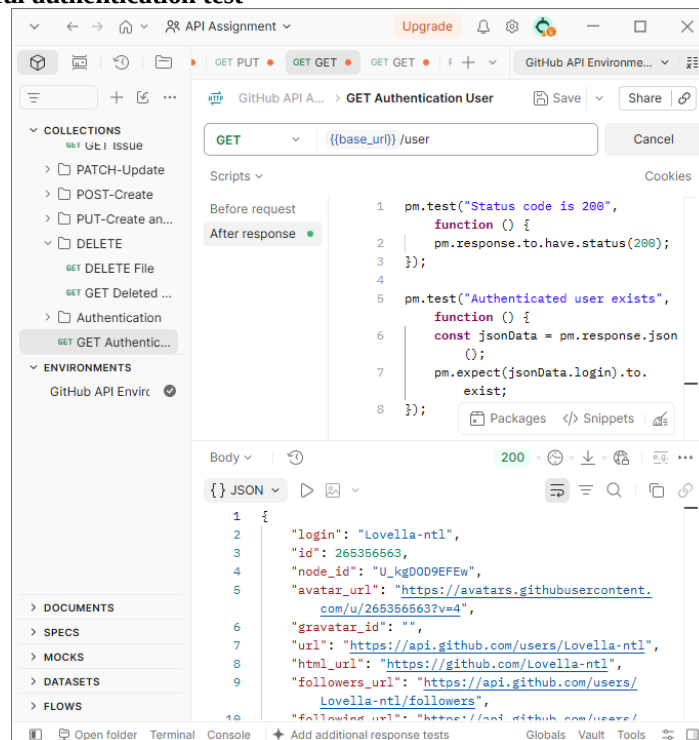
3. Testing Authentication

After setting up authentication, I tested whether my authentication was working correctly.

I sent a request to GitHub's API to retrieve information about the authenticated user. The successful response showed that GitHub recognized my authentication details.

The response confirmed that my token was working and that I could continue with the other API operations.

Screenshot 2 – Successful authentication test



4. Getting Repository Information

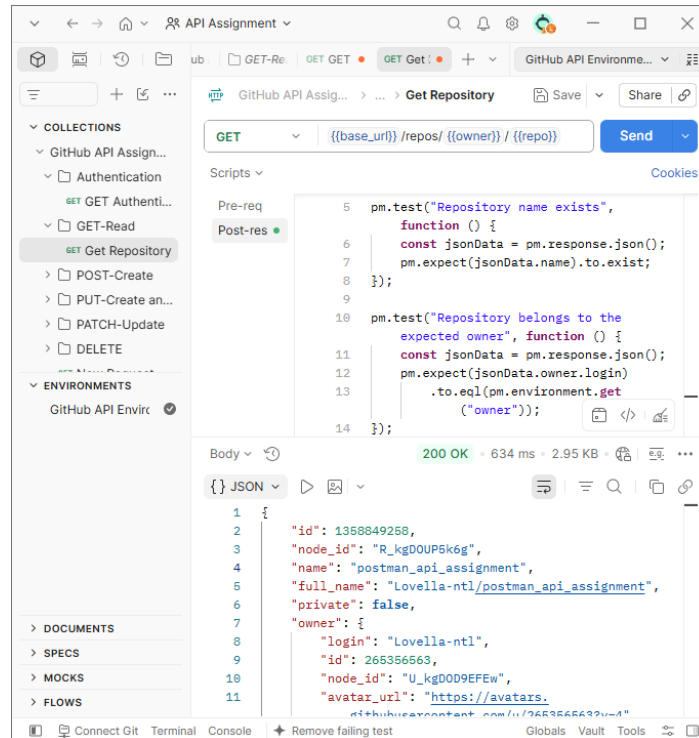
Next, I used a GET request to retrieve information about my GitHub repository.

The GET method is used to retrieve existing information without changing the resource. I entered the repository owner and repository name in the API endpoint and sent the request.

The response returned information about the repository, such as its name, owner, visibility, URL, and other repository details.

This confirmed that I could successfully access my GitHub repository through the API.

Screenshot 3 – GET repository request and response



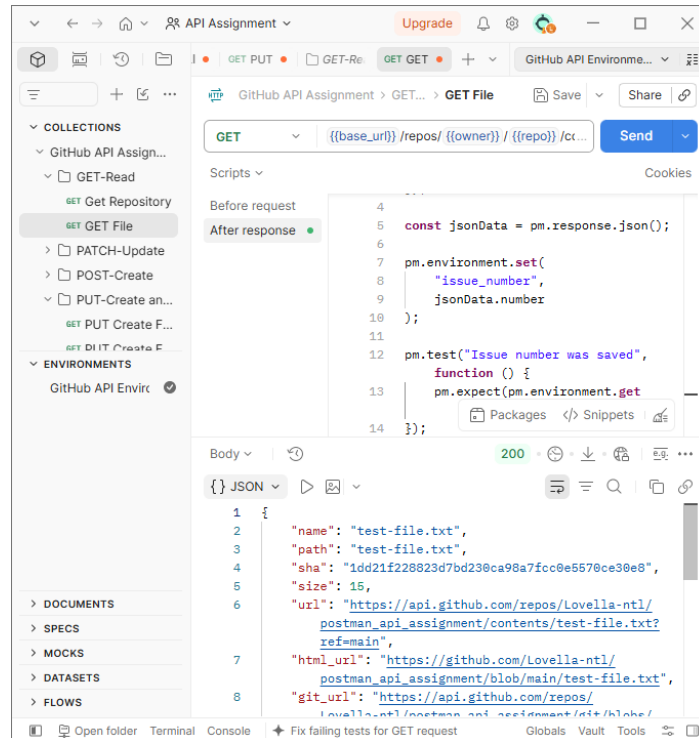
5. Getting a File from the Repository

After retrieving the repository information, I used another GET request to retrieve a specific file from the repository.

I provided the repository details and the path of the file that I wanted to access. GitHub returned information about the file, including its name, path, size, and content information.

This allowed me to verify that I could use the API to access individual files stored inside my repository.

Screenshot 4 – GET file request and response



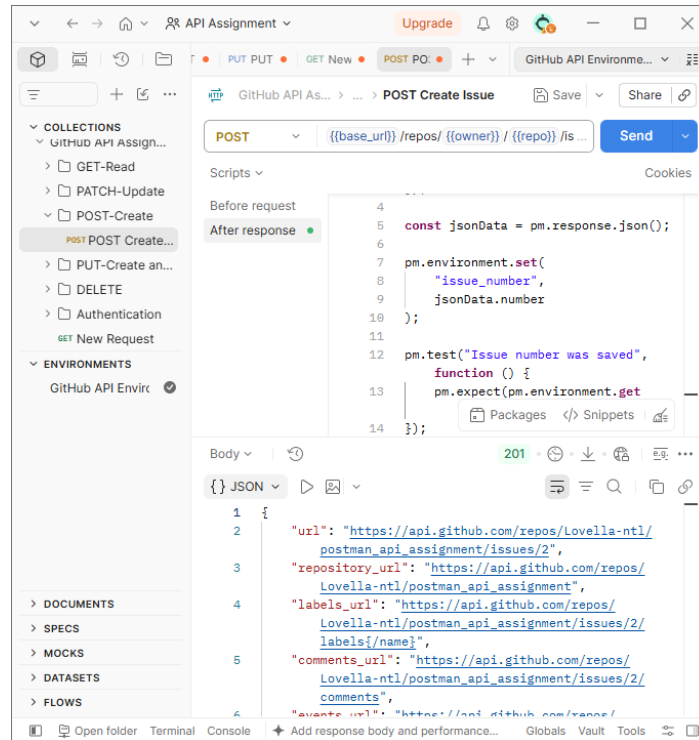
6. Creating an Issue Using POST

I then used the POST method to create a new issue in my GitHub repository.

A POST request is normally used when creating a new resource. In this case, I provided the required information for the issue, such as its title and description.

After sending the request, GitHub returned a successful response containing information about the newly created issue. This confirmed that the API request had successfully created the issue.

Screenshot 5 – POST request creating an issue



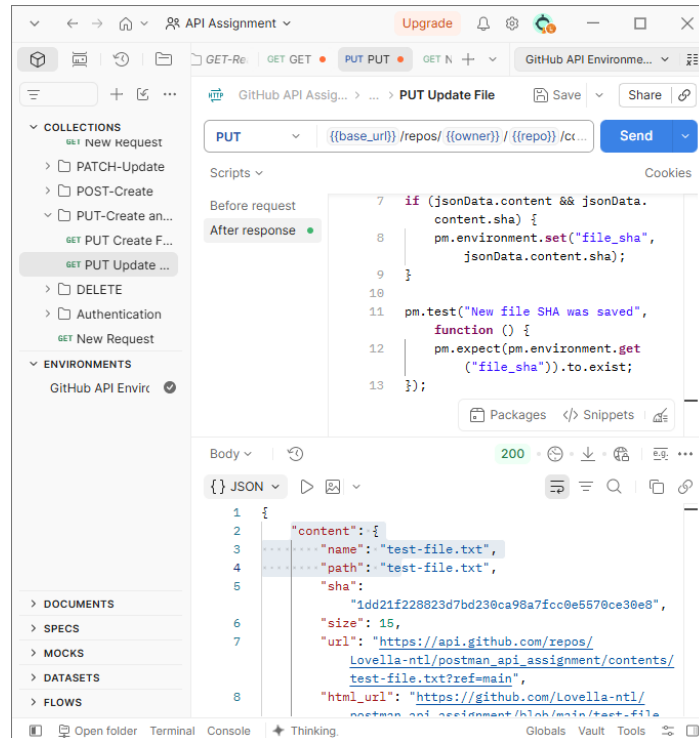
7. Creating or Updating a File Using PUT

Next, I used the PUT method to create or update a file in the repository.

For this request, I specified the file path and provided the required file content. GitHub also requires a commit message when creating or updating repository files through the API.

The successful response showed that the file operation had been completed and that a commit was created. This demonstrated how the API can be used to make changes to files in a GitHub repository.

Screenshot 6 – PUT request creating/updating a file



8. Updating an Issue Using PATCH

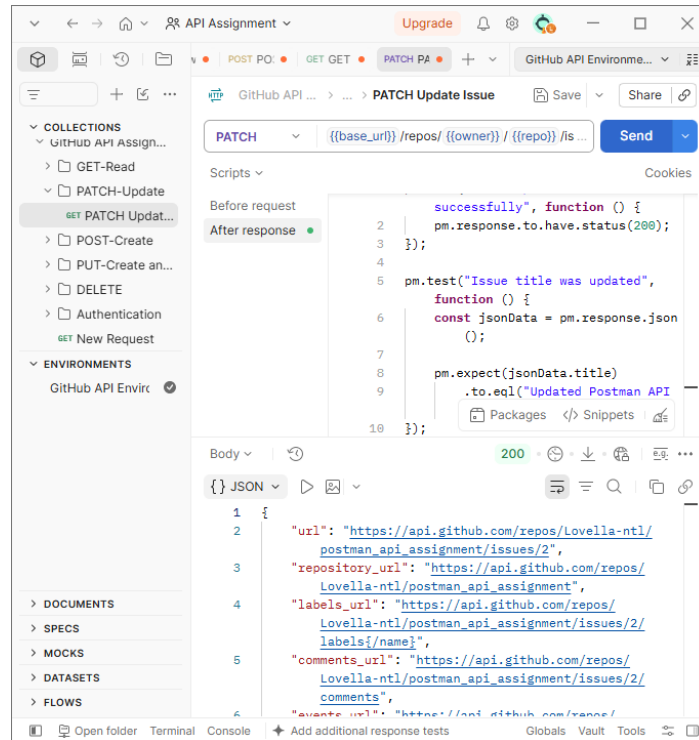
After creating an issue, I used the PATCH method to update it.

PATCH is used when making changes to an existing resource. I selected the issue that I wanted to modify and changed the required information, such as the issue title, body, or other available properties.

After sending the request, GitHub returned the updated issue information. This confirmed that the issue had been successfully modified.

This step helped me understand the difference between retrieving a resource and modifying an existing resource through an API.

Screenshot 7 – PATCH request updating an issue



9. Deleting a File Using DELETE

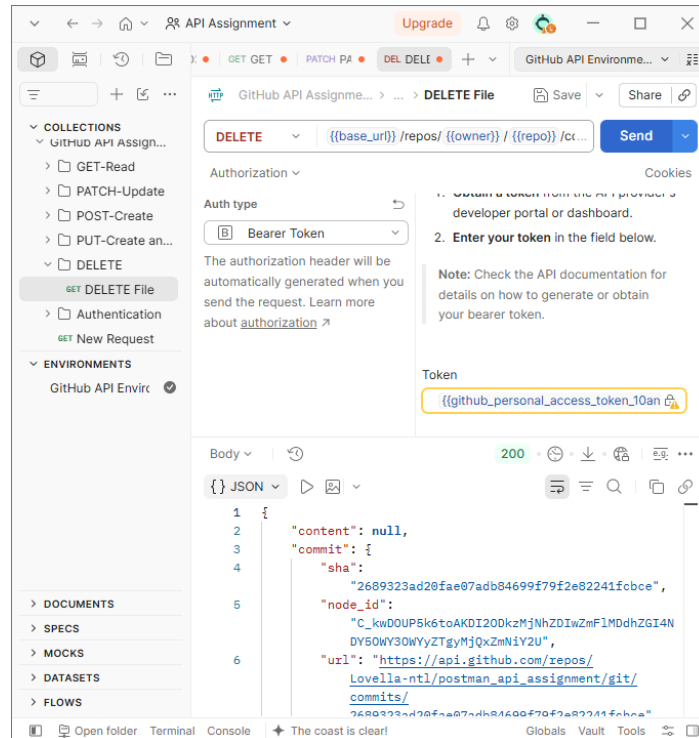
I also tested the DELETE method by deleting a file from the repository.

Before sending the request, I specified the file that I wanted to remove and provided the required information, including the file's current SHA and a commit message.

After sending the request, GitHub returned a successful response indicating that the file had been deleted.

This demonstrated how the DELETE HTTP method can be used to remove an existing resource through an API.

Screenshot 8 – DELETE request and successful response



10. Using Environment Variables

To make my Postman collection easier to manage, I created and used environment variables.

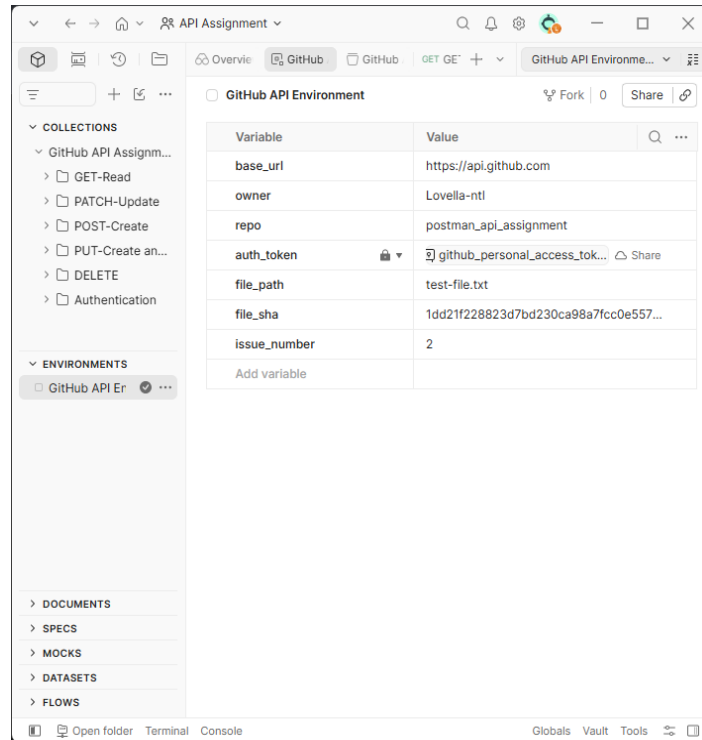
Instead of repeatedly entering values such as my GitHub username, repository name, and authentication token, I stored them as variables in the Postman environment.

I then used the variables inside my API requests. For example, instead of writing the repository name directly in every endpoint, I could use a variable such as `{{repo_name}}`.

Using environment variables made my requests more organized and reduced the amount of information that I needed to enter manually.

It also made it easier to reuse the same collection when working with different values.

Screenshot 9 – Postman environment variables



11. Running the Collection

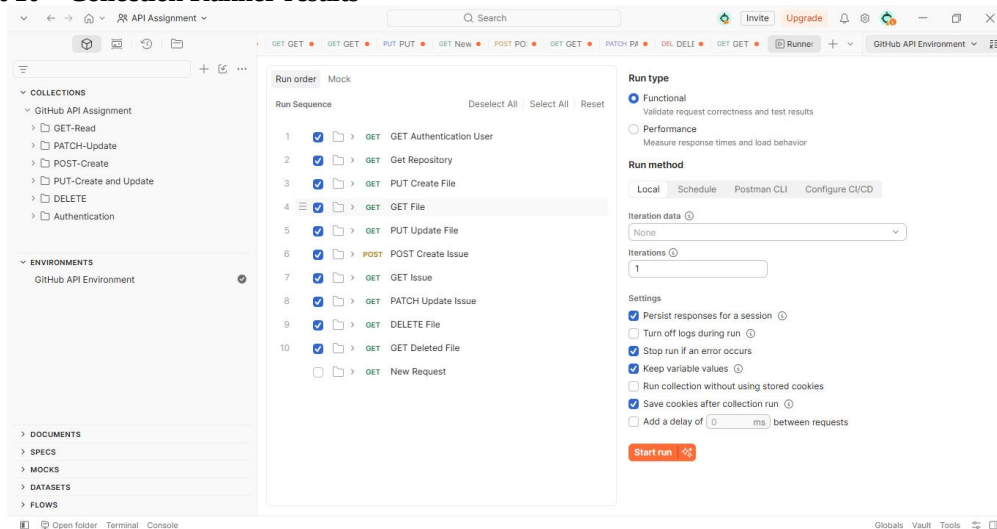
Finally, I used the Postman Collection Runner to run multiple API requests from my collection.

The Collection Runner allowed me to execute the requests in sequence instead of manually running each request one by one.

I selected my collection and the appropriate environment before starting the run. Postman then executed the requests and displayed the results, including which requests were successful and whether any tests failed.

This helped me verify that my API requests were working together as a collection and made the testing process more efficient.

Screenshot 10 – Collection Runner results



[NOTE: this shows the Runner configured with all 10 requests queued. The actual results screenshot available showed a run error (empty request URL) rather than passing tests — replace with a successful run screenshot before submitting.]

12. Additional API Tests

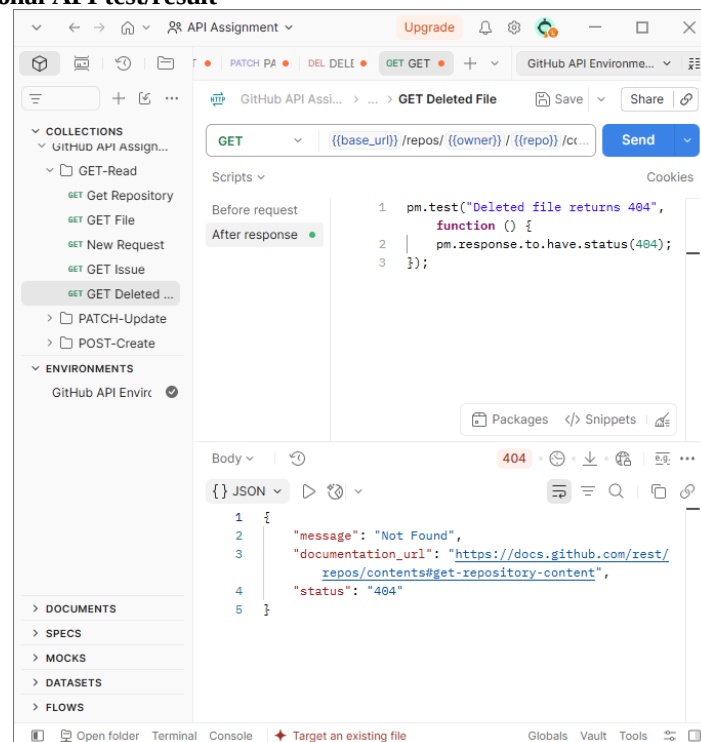
In addition to the main requests, I performed additional checks to make sure that the API operations were working correctly.

I checked the response status codes and response bodies returned by GitHub. Successful requests generally returned status codes such as 200 OK, 201 Created, or 204 No Content, depending on the operation.

I also checked the returned data to make sure that the requested repository, file, or issue was correctly identified and that changes were successfully applied.

These tests helped me confirm that my API requests were not only being sent successfully but were also producing the expected results.

Screenshot 11 – Additional API test/result



13. Summary of HTTP Methods Used

During the assignment, I used several HTTP methods for different purposes:

- GET — Retrieve repository and file information
- POST — Create a new GitHub issue
- PUT — Create or update a repository file
- PATCH — Update an existing issue
- DELETE — Delete a repository file

Each method had a different purpose. GET was mainly used to retrieve information, while POST, PUT, PATCH, and DELETE were used to create, modify, or remove resources.

14. Conclusion

Through this assignment, I gained practical experience using the GitHub REST API with Postman. I learned how to authenticate API requests and how to use different HTTP methods to interact with GitHub resources.

I successfully worked with repositories, files, and issues using GET, POST, PUT, PATCH, and DELETE requests. I also learned how environment variables can make API collections easier to manage and how the Collection Runner can be used to test multiple requests efficiently.

Overall, this assignment helped me understand how APIs communicate with applications and how tools such as Postman can be used to test and manage API requests in a practical way.